

Protocol Specification

LLM-Assisted Reliable File Transfer over UDP

1. Overview

This project implements a reliable file-transfer protocol on top of UDP, built from scratch rather than reusing an existing reliable-transport implementation. The reliability layer uses a sliding-window design with cumulative acknowledgements and Go-Back-N retransmission, parameterised by window size. For the submitted build the window size is set to 1, which makes the protocol behave as stop-and-wait; this configuration was chosen to prioritise correctness and thorough testing over throughput optimisation, and it required no redesign, since stop-and-wait is simply the `windowSize = 1` case of the same sliding-window logic. Both configurations are implemented, tested, and demonstrable.

The protocol has two message families. `START`, `START_ACK`, `FINISH`, `FINISH_ACK` and `ERROR` are infrequent control messages, encoded as JSON. `DATA` and `ACK` are high-frequency and use a compact binary header instead, described in the next section. A receiver distinguishes the two families on arrival by inspecting the first byte of the datagram: a JSON message always begins with `'{'` (0x7B), which never appears as a `MessageType` ordinal (0–6), so no additional framing is required.

2. Packet Formats

`DATA` and `ACK` packets share one fixed-size 27-byte header. An `ACK` packet has `payloadLen = 0` and carries no payload bytes.

Field	Size	Type	Meaning
<code>type</code>	1 byte	unsigned byte	<code>MessageType</code> ordinal; only <code>DATA</code> or <code>ACK</code> appear here
<code>transferId</code>	16 bytes	raw UUID	UUID split into two 8-byte longs (most/least significant bits)
<code>seqNum</code>	4 bytes	int	Chunk index for <code>DATA</code> ; cumulative ACK number for <code>ACK</code>
<code>payloadLen</code>	2 bytes	unsigned short	Payload bytes following; 0 for <code>ACK</code>
<code>crc32</code>	4 bytes	unsigned int	CRC-32 over every field above, including payload
<code>payload</code>	0-1024 bytes	raw bytes	File chunk data; absent for <code>ACK</code>

All multi-byte fields use big-endian (network) byte order, the standard Internet convention and Java's `DataOutputStream` default. Maximum `DATA` packet size is $27 + 1024 = 1051$ bytes; an `ACK` is always exactly 27 bytes.

Fragmentation avoidance. The 1024-byte payload cap is a deliberate simplicity choice. A maximum-size `DATA` packet, plus 8 bytes of UDP header and 20 bytes of IPv4 header, totals 1079 bytes, comfortably under the standard 1500-byte Ethernet MTU, leaving over 400 bytes of headroom. This avoids IP fragmentation entirely, at the cost of needing more packets to move a large file than a bigger chunk size would require.

Sequence numbers. `seqNum` is chunk-indexed (0, 1, 2, ...), not byte-offset: every chunk except possibly the last is exactly `chunkSize` bytes, so byte precision adds nothing here. An `ACK`'s `seqNum` is cumulative: "everything through this chunk has been received with no gaps," which is what allows one `ACK` to advance the sender's window past several chunks at once when `windowSize > 1`.

CRC-32 scope. The checksum covers every header field except itself, plus the payload, and this coverage applies to `ACK` packets as well as `DATA`. A corrupted `ACK` is a genuine correctness risk, not just noise: a bit-flip in its `seqNum` could make the sender believe the wrong chunk had been acknowledged. A failed-CRC `ACK` is discarded and treated exactly like a lost `ACK`, left to the timeout/retransmission logic described in Section 4.

Framing validation. Beyond the exact-length and CRC checks above, the receiver checks each DATA sequence against the expected length implied by the agreed chunk size, including the shorter final chunk and the single zero-length chunk used to represent an empty file. This was added after focused testing surfaced a framing gap in an earlier build; it is now covered by passing focused tests and independent checks against a corrected reconstructed-file hash.

3. State Machine

3.1 Sender

IDLE → send START → AWAITING_START_ACK. On acceptance, the sender enters SENDING: it fills the window up to windowSize with new DATA packets, starts a timer for each, and waits (bounded by the socket's receive timeout) for an ACK. A valid ACK advances the window's base and cancels the timers of the newly-acknowledged sequence numbers. Independently, any packet whose timer has elapsed triggers a Go-Back-N retransmission of every sequence number still in flight. If the retry limit is exceeded with no progress, the transfer moves to FAILED. Once every chunk is acknowledged, the sender computes the whole-file SHA-256, sends FINISH, and moves to AWAITING_FINISH_ACK; a verified FINISH_ACK moves it to COMPLETE, an unverified one to FAILED.

3.2 Receiver

IDLE → receive START → VALIDATING_START, which checks that the proposed chunkSize falls within (0, MAX_PAYLOAD_SIZE]; an invalid value is rejected via START_ACK(accepted = false) and the transfer fails immediately. On acceptance the receiver enters RECEIVING and loops on incoming datagrams, branching on the first byte as described in Section 1. A DATA packet is decoded and CRC-checked; an invalid packet is discarded silently. A valid packet is handed to the sequence tracker, which reports one of three outcomes: ACCEPTED (write the payload to the correct file offset and ACK it), DUPLICATE (already have it; re-ACK without rewriting, since the sender's original ACK may simply have been lost), or OUT_OF_ORDER_DISCARDED (a gap exists before it; discard silently and send no ACK, consistent with the Go-Back-N design in Section 4, which does not buffer out-of-order data). A FINISH message triggers completeness and hash verification, and the outcome is reported via FINISH_ACK.

Known limitation: *START and FINISH are now bounded retry handshakes rather than single blocking receives (Section 4 gives the exact timing), so one lost START_ACK or FINISH_ACK no longer stalls the transfer. One gap remains: the file size advertised in START is converted to an int chunk count with no explicit extreme-size overflow bound, so experiments should use bounded, practical file sizes. This is a documented limit, not a blocker observed in any run to date, stated here rather than left implicit.*

4. Timeout and Retry Policy

Fixed, not adaptive. Every in-flight DATA packet is governed by a single configurable timeout value (default 200 ms), tracked independently per packet rather than once per window. This is necessary for windowSize > 1, since chunks in the same window are sent at different real moments and must be judged against the timeout independently. An adaptive RTO scheme (e.g. Karn's algorithm) was considered and deliberately set aside: it is meaningful additional complexity for a benefit that is difficult to demonstrate rigorously inside this project's timeline, whereas a fixed timeout's weaknesses are directly observable and worth discussing with evidence in the evaluation report.

200 ms is a baseline, not a claimed-optimal value. On a development machine over loopback, real round-trip time is sub-millisecond, so almost any timeout value "works" and proves nothing. The genuine test of whether 200 ms is reasonable only becomes possible once controlled network impairment (delay/jitter) is introduced experimentally; at that point, measured RTT is compared against 200 ms to check for excessive spurious retransmission on the one hand, or unnecessarily slow loss recovery on the other. That comparison belongs in the evaluation report, not asserted here in advance.

Retransmission strategy: Go-Back-N. When the oldest in-flight sequence number times out, every sequence number currently in flight is retransmitted, not only the one that timed out, and all of their timers restart. A chunk that in fact arrived safely may still be resent if a chunk before it was lost, a known inefficiency accepted in exchange for a receiver

that never needs to buffer out-of-order data. At `windowSize = 1` this collapses to ordinary stop-and-wait retransmission of the single outstanding packet.

Retry limit. The controller counts consecutive retransmission rounds that make no progress (rounds in which no ACK has advanced the window since the previous round) and resets that counter the moment real progress occurs. Exceeding the configured limit (default 5) fails the transfer permanently. Counting no-progress rounds rather than total retransmissions matters: a long transfer over a genuinely lossy link may need many retransmissions while still making steady progress, and should not be penalised for that; the limit exists to detect a stuck connection, not an ordinary one.

Handshake retries (START and FINISH). START and FINISH each use their own bounded retry policy, separate from the per-packet DATA/ACK timer above. By default the sender waits 1000 ms per START attempt and permits five retries (2000 ms per attempt in the validated command console), accepting only a START_ACK from the expected receiver; FINISH uses the same 1000 ms/five-retry pattern and accepts only a FINISH_ACK carrying an explicit verified result for the active transfer. On the receiver side, an initial START may arrive up to five minutes after the receiver starts, and once DATA transfer is under way the receiver fails the transfer if 60 seconds pass with no new DATA progress. After sending its first FINISH_ACK, the receiver keeps that cached result for a bounded completion-recovery window (6250 ms by default) so a retransmitted FINISH from the same transfer gets the same answer without rewriting or rehashing the file.

5. LLM Validation Boundary

The LLM never controls execution directly. A natural-language request is converted by the LLM into a strict JSON command, which a deterministic Java validator checks before anything is allowed to run. The validator, not the LLM, is the sole authority over whether an action proceeds.

User → LLM (proposes JSON) → CommandValidator → TransferController → engine → metrics → LLM (explains measured results)

Three structured command types are recognised: `start_transfer` (`file_id`, `receiver_id`, `window_bytes`, `timeout_ms`), `status` (`transfer_id`) and `explain` (`run_id`, `question`); anything else is rejected as an unsupported request before it reaches the validator. The parser itself is strict JSON: unexpected or duplicate argument names, missing required fields, and wrong-typed values (a non-integer `window_bytes`, a non-UUID `transfer_id`) are all rejected as malformed commands rather than coerced. The validator then checks every field the LLM cannot be trusted to bound itself. `file_id` and `receiver_id` are never raw paths, hosts or ports supplied by the model; they are looked up against Java-configured allowlists (an approved-files map and an approved-receivers map), so the model can only ever select a pre-approved source file and a pre-approved destination address and port, never invent one. Chunk size is fixed by Java at 1024 bytes and is not model-configurable at all. `window_bytes` must fall within 1024-1,048,576 bytes (rounded down to whole packets) and `timeout_ms` within 50-5,000 ms; both fall back to Java defaults when omitted. The retry limit is fixed by Java and cannot be set by a command. A dispatcher also enforces transfer lifecycle state: only one transfer may be active at a time, and a request that has already been dispatched cannot be re-dispatched by a retried or duplicated model call.

Provider note. The natural-language interpretation and explanation steps currently call OpenAI's gpt-5-mini over the Responses API, chosen pragmatically for response quality and integration effort within the project timeline. This is an implementation detail, not a weakening of the boundary above: every check described in this section runs in Java regardless of which model proposed the command, and a locally-run or other free model could be substituted behind the same ExplanationClient/GptClient interfaces without changing the validator. The cost and privacy trade-offs of this choice, and the free/local alternatives considered, are discussed in the evaluation report.

6. Sequence Diagram

The exchange below illustrates a normal handshake, windowed data transmission (`windowSize = 3`), one packet loss with its resulting timeout and Go-Back-N retransmission, and a verified completion.

